

MERCATURA QT WALLET

Beginner's Guide to Creating, Backing Up, Receiving and Sending MCA

Public Testnet Edition — written for first-time users

Start here

Mercatura Core / mercatura-qt is both a full validating node and a desktop wallet. This guide covers the wallet side: creating a normal Mercatura post-quantum wallet, protecting it with a passphrase, backing it up, receiving test MCA, sending transactions, restoring a backup, and understanding the most common warnings.

TESTNET ONLY. Use only test MCA while following this guide. Testnet coins have no monetary value. Testnet native addresses begin with `tmca`; mainnet uses a different network and `mca` addresses.

The four rules to remember

- Back up the wallet before you care about the funds, and make a new backup after encrypting it.
- If you forget the encryption passphrase, the wallet cannot spend its funds for you.
- Before sending, check the first and last characters of the destination and confirm that it belongs to the correct network.
- Wait for the node to synchronize. A wallet that is far behind the chain may show incomplete balances or transaction history.

What makes Mercatura's wallet different

Normal Mercatura ownership uses ML-DSA-65 post-quantum authorization through native witness-v2 addresses. The Qt wallet hides the multi-kilobyte public keys and signatures from normal users; your everyday workflow still looks familiar: create a wallet, generate an address, receive, send, encrypt and back up.

1. Before creating the wallet

The safest wallet setup starts with a correctly synchronized Mercatura testnet node.

1.1 Confirm you are on testnet

- The application should clearly identify the test network.
- A normal testnet receive address begins with `tmca` (native PQ addresses normally appear as `tmca1z...`).
- Do not use an `mca...` mainnet address in this guide.
- If the node has zero peers, follow the separate Mercatura Full Node Guide and configure the current public testnet bootstrap peer.

1.2 Wait for synchronization

Mercatura Core must know the current chain before it can show a complete wallet state. Let the synchronization progress reach the public testnet tip. If you are unsure, open the node window/console and compare `getblockchaininfo` with the current Explorer or another trusted node.

1.3 Wallet and network are separate concepts

Thing	What it does
Full node	Downloads and verifies blocks; connects to peers; maintains chain state.
Wallet	Holds the private wallet state needed to control your addresses and sign spends.
Qt application	Runs both the full node interface and the wallet interface in one desktop program.

1.4 What to prepare

- A unique wallet name, such as `test-wallet-1`.
- A strong wallet passphrase that you can reliably recover.
- A safe location for a backup file — ideally removable/offline storage plus a second independent copy.
- A small amount of test MCA from a faucet, another tester, or a mined/distribution wallet.

No ordinary BIP39-style recovery phrase is assumed here. Mercatura's PQ wallet keeps its deterministic PQ master material and derivation metadata inside the wallet architecture. The normal user recovery workflow is the supported wallet backup/restore path. Do not expect an `xprv/xpub` or invented BIP32-style PQ key.

2. Create a normal Mercatura wallet

For an ordinary spending wallet, use the defaults. Do not enable advanced wallet modes unless you know exactly why you need them.

2.1 Open the Create Wallet dialog

1. Start Mercatura Core.
2. Click File.
3. Click Create Wallet...
4. Enter a simple wallet name. Example: test-wallet-1.

2.2 Recommended choices

Create Wallet option	Beginner choice	Why
Encrypt Wallet	YES — recommended	Protects the wallet with a passphrase.
Disable Private Keys	NO	A normal spending wallet needs its private/PQ signing material.
Make Blank Wallet	NO	This is an advanced import/descriptor workflow.
External signer	NO	Do not select unless a specifically supported external signer is being tested.

Then click **Create**.

2.3 Choose the passphrase

If you selected Encrypt Wallet, Mercatura Core will ask for a passphrase. Use a long passphrase that you do not reuse elsewhere. Type it carefully twice. Record it securely before moving on.

Passphrase loss is serious. Encryption is meant to prevent someone who steals the wallet file from immediately spending. It also means you need the passphrase when the wallet must unlock to sign. A backup file does not magically reveal a forgotten passphrase.

2.4 If you created an unencrypted wallet

You can encrypt it later from **Settings** → **Encrypt Wallet...**. If you do that, immediately replace your old unencrypted backup with a fresh backup of the encrypted wallet.

2.5 Confirm the wallet is loaded

The wallet selector should show your new wallet. The Overview, Send, Receive and Transactions pages should now be available. At this point the balance may still be zero — that is normal.

3. Back up the wallet before receiving funds

A wallet that has never been backed up is not ready to hold funds you care about.

3.1 Create the backup

5. Click File.
6. Click Backup Wallet...
7. Choose a location that is not inside the active Mercatura data directory.
8. Give the file a clear name such as mercatura-test-wallet-1-backup.dat.
9. Finish the save and confirm the file actually exists.

3.2 Make two copies

For anything important, keep at least two independent backup copies. A practical pattern is one offline USB/storage copy and one second copy in another secure location. Do not leave your only backup on the same disk as the active wallet.

3.3 Encrypt first or back up again

If you encrypt a wallet after making an earlier unencrypted backup, create a new backup immediately. The Qt wallet explicitly warns that previous unencrypted backups should be replaced after encryption because the newly encrypted wallet state is the one you should preserve going forward.

3.4 What the backup is protecting

Mercatura's wallet backup preserves the authoritative PQ wallet state required to recover deterministic ownership. Expanded ML-DSA secret-key material is derivable/cached material; users should not have to export thousands of large raw keys individually.

Do not rename the active wallet database while Mercatura Core is running. Use File → Backup Wallet... for normal backups. Manually copying live wallet files is unnecessary and easier to get wrong.

4. Receive test MCA

Receiving is the safest first transaction. It proves your wallet can generate a native Mercatura PQ destination and recognize funds sent to it.

4.1 Generate a receive request

10. Click Receive.
11. Optionally enter a Label, such as From Alice or Test faucet. The label is local to your wallet.
12. You may leave the requested amount blank if you only need an address.
13. Click the button to create/request the payment information.
14. Copy the generated address or use the QR code.

4.2 Check the address before sharing it

On public testnet, a normal native Mercatura PQ address should begin with **tmca** (commonly **tmca1z...** for witness-v2 PQ authorization). If it looks like a mainnet mca... address or another cryptocurrency address, stop and investigate.

4.3 Share only the address / QR code

A receive address is public information. It is fine to send it to another tester. Never send someone your wallet backup, passphrase, internal private data, or any debug output that contains secrets.

4.4 What happens after someone sends

Status	Meaning
Unconfirmed / pending	The transaction has been seen but is not yet in a block.
Confirmed	A block includes the transaction. Additional blocks increase confirmation depth.
Conflicted	A reorg or replacement/conflict invalidated the wallet's earlier view of the transaction.

Mercatura targets 150-second blocks, but actual block arrival varies. Do not interpret a few minutes without a confirmation as automatic failure.

Mining rewards are different. If the funds came directly from a coinbase/mined output, coinbase maturity applies before the output can be spent. Ordinary received transactions do not have that special maturity wait.

5. Send test MCA safely

For your first send, use a small amount and let the wallet choose inputs and change automatically.

5.1 Basic send

15. Click Send.
16. Paste the recipient's testnet address into Pay To / recipient address.
17. Confirm the address is a Mercatura testnet destination — normally tmca...
18. Enter the amount using MCA with two decimal places.
19. Optionally add a label for your own address book/history.
20. Leave coin control and custom change settings alone for your first transaction.
21. Review the displayed fee and total carefully.
22. Click Send and read the confirmation dialog before approving.
23. If the wallet is encrypted, enter the passphrase when prompted to unlock for signing.

5.2 Mercatura fee behavior

Mercatura uses 0.01 MCA as one base unit. The locked size policy is 0.01 MCA per started 1,000 effective bytes, with a minimum transaction fee of 0.01 MCA unless estimation requires more. PQ inputs are much larger than classical Bitcoin-style signature inputs, so a normal transaction can legitimately cost more than 0.01 MCA.

Rule	Value
Monetary precision	1 MCA = 100 base units
Minimum transaction fee	0.01 MCA
Size rule	0.01 MCA per started 1,000 effective bytes
Recommended dust threshold	0.02 MCA
PQ input estimator checkpoint	5,309 fee bytes / 5,432 WU per PQ input

5.3 Change is normal

If your selected wallet input is larger than the payment plus fee, the unused amount returns to a newly generated wallet-controlled change address. Do not manually set a custom change address unless you understand coin control.

Always read the final confirmation. Check recipient, amount and fee again. Cryptocurrency payments are not reversed by a bank or help desk if you send to the wrong valid address.

6. Encryption, locking, passphrases and privacy

6.1 What encryption protects

Wallet encryption protects the private signing material at rest. When the wallet needs to sign a spend, it must be unlocked with the correct passphrase. Receiving funds does not require the wallet to remain unlocked.

6.2 Change the passphrase

Use **Settings** → **Change Passphrase....** You will need the existing passphrase and the new passphrase. After a passphrase/security change, making another fresh backup is a sensible operational habit.

6.3 Mask Values

The Settings menu includes a privacy/mask-values option. Use it when sharing your screen or demonstrating the wallet in public. It hides visible amounts; it does not encrypt the wallet or hide blockchain activity.

6.4 Labels are not ownership

Labels are local organizational notes. Giving an address the label 'Savings' or 'Alice' does not change cryptographic ownership and does not synchronize that label to other wallets.

6.5 Things not to share

- Wallet backup files.
- Wallet encryption passphrase.
- Any exported secret/private wallet data.
- Screenshots that accidentally show sensitive file paths or secrets.
- RPC credentials or authentication cookies.

Testnet habits become mainnet habits. Treat the test wallet as practice for mainnet operational discipline. The coins are disposable; the security habits should not be.

7. Restore a wallet and use multiple wallets

7.1 Restore from a backup

24. Start Mercatura Core on the correct network.
25. Click File.
26. Click Restore Wallet...
27. Select the .dat wallet backup file.
28. Choose a wallet name for the restored copy when prompted.
29. Let the wallet load and rescan/reconcile chain history if required.
30. Generate a new receive address and verify old known addresses/balances are recognized as expected.
31. Before relying on the restore, make a small test spend.

Mercatura's PQ backup/recovery workflow has been tested with restoration on a separate node/data directory and with older backups followed by deterministic PQ regeneration/rescan. That is why the backup file—not manual copying of large ML-DSA keys—is the normal recovery path.

7.2 Open or close wallets

The File menu supports Open Wallet, Close Wallet and Close All Wallets. Mercatura Core can load more than one wallet at the same time. Always check the wallet selector before receiving or sending so you know which wallet is active.

7.3 Create a second test wallet

A useful test is to create Wallet A and Wallet B, send $A \rightarrow B$, then $B \rightarrow A$. Each wallet should see only its own spendable outputs, even if you reuse the same text label in both.

7.4 Duplicate restored wallets

Be careful with clones. Restoring the same wallet backup under another name can create two loaded wallet copies that control the same deterministic funds. This is useful for testing but confusing in normal use. Do not treat two copies of the same wallet as independent ownership.

7.5 Mainnet later

When mainnet launches, use a mainnet-specific wallet and backup discipline. Do not assume testnet addresses, balances or chain data transfer to mainnet. Native mainnet Mercatura addresses use the mca HRP, while testnet uses tmca.

8. Transactions, troubleshooting and common mistakes

8.1 Transaction history

Click Transactions to see wallet history. The Qt wallet should show pending/confirmed/conflicted states, amounts and transaction details. If a reorganization changes confirmation state, the wallet history should update rather than pretending the old confirmation is permanent.

8.2 Common problems

Problem	Check first
Balance looks wrong	Is the node fully synchronized? Is the correct wallet selected?
Address rejected	Is it the correct network? Testnet normally uses tmca...
Cannot send	Check available balance, wallet lock state, coinbase maturity, and fee.
Passphrase rejected	Check capitalization/keyboard layout. Repeated guessing does not recover a forgotten phrase.
Fee seems high	PQ authorization is large; review selected inputs and displayed effective size/fee.
Transaction stays pending	Check peers, mempool acceptance, and whether blocks are being mined.
Wallet missing after restart	Use File → Open Wallet and select the existing wallet.
Restore shows old history only	Allow the restore/rescan to complete; verify node sync first.

8.3 Node window / console

The Window menu provides the Node window for diagnostics. The console is powerful; beginners should use read-only commands unless following a specific trusted procedure.

```
getblockchaininfo
getnetworkinfo
getwalletinfo
getbalances
listtransactions
```

Do not paste random console commands from strangers. The debug console can perform wallet and node actions. Treat copied commands with the same caution you would treat executable code.

8.4 Wrong-network protection

A mainnet address supplied to a testnet send should be rejected rather than silently remapped. If an address looks unfamiliar or a send dialog behaves ambiguously, cancel the transaction and verify the network/address before proceeding.

9. Advanced features — what beginners should leave alone

Mercatura inherits many Bitcoin Core wallet tools. Some are useful, some are specialized, and some classical-ownership concepts do not map directly to Mercatura's PQ model.

Feature	Beginner guidance
Coin Control	Leave off unless you specifically need to choose UTXOs.
Custom change address	Do not use for normal sends. Let the wallet generate PQ change automatically.
PSBT	Supported and important for advanced workflows, but unnecessary for ordinary send/receive.
Load PSBT from file/clipboard	Use only when following an intentional PSBT procedure.
Blank/watch-only wallet options	Advanced. Do not select for a normal spending wallet.
External signer	Do not assume generic hardware-wallet support; use only with a specifically validated Mercatura setup.
Legacy/classical address choices	Normal Mercatura ownership is native PQ. Classical ECDSA/Schnorr ownership is intentionally not the normal spend path.
Sign/Verify Message	Inherited UI may not represent the native PQ transaction-authorization model; do not use it as a funds-safety or ownership test unless specifically validated.

10. Final beginner checklist

- ☐ Mercatura Core is on testnet and fully synchronized.
- ☐ My normal receive address begins with tmca.
- ☐ I created a normal wallet without advanced blank/watch-only options.
- ☐ The wallet is encrypted with a passphrase I can recover.
- ☐ I made a fresh .dat wallet backup after encryption.
- ☐ I have at least two independent backup copies for anything important.
- ☐ I successfully received a small testnet transaction.
- ☐ I successfully sent a small testnet transaction and understood the fee.
- ☐ I know that change addresses are normal and are wallet-controlled.
- ☐ I know how to restore the wallet from File → Restore Wallet... .
- ☐ I know that the debug console is powerful and should not be used casually.

You are ready. If the node is synchronized, your wallet is encrypted and backed up, you can receive to a tmca address, and you can send a small confirmed test transaction, you have completed the normal Mercatura Qt wallet workflow.

Technical basis: current Mercatura Qt menu/action source, Mercatura native-PQ wallet functional tests, backup/restore tests, Mercatura Wallet Compatibility Test Guide, and the current locked fee/address policy. This document focuses on the supported normal user workflow; advanced inherited Bitcoin Core surfaces are intentionally treated conservatively where Mercatura-specific behavior may require separate qualification.